

Algoritmi e Strutture Dati – 26/8/2024

Esercizio A1 – Complessità – Punti ≥ 8

Le tre funzioni di ricorrenza possono essere scritte in questo modo:

$$\begin{aligned}T_A(n) &= 5T_A(n/2) + n \\T_B(n) &= 2T_B(n-2) + 1 \\T_C(n) &= 9T(n/3) + n^2\end{aligned}$$

Utilizzando le varie versioni del teorema dell'esperto, otteniamo:

$$\begin{aligned}T_A(n) &= \Theta(n^{\log_2 5}) \\T_B(n) &= \Theta(2^n) \\T_C(n) &= \Theta(n^2 \log n)\end{aligned}$$

Ovviamente, l'algoritmo B è da escludere; l'esponente di $T_A(n)$ è maggiore di 2, quindi l'algoritmo C è quello che ha complessità inferiore.

Esercizio A2 – Alberi isobati – Punti ≥ 10

Si noti che se un albero è isobato, allora i sottoalberi radicati nei figli della radice sono anch'essi isobati. Per assurdo, supponiamo che non sia così; allora in uno di essi ci sarebbero due foglie con profondità p_1 e p_2 nel sottoalbero, con $p_1 \neq p_2$. Queste due foglie avrebbero profondità $p_1 + 1 \neq p_2 + 1$ nell'albero considerato, rendendolo non isobato, il che è assurdo.

Si potrebbe calcolare la profondità delle foglie utilizzando un parametro aggiuntivo che viene incrementato a ogni chiamata; per semplificare, dato un nodo t , scriviamo una funzione ausiliaria che, applicata a un nodo t , restituisce:

- la distanza dalle sue foglie, se è isobato;
- -1 se il sottoalbero radicato è vuoto, ovvero $t = \mathbf{nil}$;
- $-\infty$ se il sottoalbero radicato non è isobato e quindi le sue foglie hanno distanze diverse.

A parte il caso in cui $t = \mathbf{nil}$, la funzione ausiliaria viene richiamata sul figlio sinistro e sul figlio destro, memorizzando i risultati in p_L e p_R ; dopodiché, si gestiscono i vari casi possibili. Si noti che il caso delle foglie è gestito dalla condizione $p_R = p_L$, che somma 1 al valore $p_L = -1$ e ottiene la corretta distanza 0.

```
boolean isIsobath(TREE T)
```

```
    return helper(T) ==  $\infty$ 
```

```
int helper(TREE t)
```

```
    if t == nil then
```

```
        | return -1
```

```
    else
```

```
        int pL = helper(t.left)
```

```
        int pR = helper(t.right)
```

```
        if pL ==  $-\infty$  or pR ==  $-\infty$  then
```

```
            | return  $-\infty$ 
```

% Uno dei due sottoalberi non è isobato

```
        else if pL == -1 then
```

```
            | return pR + 1
```

% Solo figlio destro

```
        else if pR == -1 then
```

```
            | return pL + 1
```

% Solo figlio sinistro

```
        else if pR == pL then
```

```
            | return pL + 1
```

% Isobato, aumento la distanza delle foglie di 1

```
        else
```

```
            | return  $-\infty$ 
```

Essendo una visita in profondità, il costo della funzione è $O(n)$.

Esercizio A3 – Where’s negative? – Punti ≥ 12

Una soluzione banale consiste in un semplice ciclo **while** che scorre tutti i valori naturali da 1 in poi, fino a incontrare il primo valore n tale per cui $f(n) < 0$. Tale algoritmo ha costo lineare e viene valutato 30%.

Per migliorare questa soluzione, possiamo utilizzare un’idea simile al metodo della bisezione per trovare gli zeri di un funzione, che prevede di considerare un intervallo $start, end$, dove $f(start) > 0$ e $f(end) < 0$, e continuare a restringere l’intervallo fino a quando si identifica il primo valore negativo. A differenza del caso continuo, il nostro algoritmo termina sempre con un valore preciso.

Non conoscendo a priori un valore end tale che $f(end)$ sia minore di zero, dobbiamo cercarlo. Utilizzeremo una ricerca esponenziale, che consiste nell’inizializzare il valore end a 1 e raddoppiarlo progressivamente finché $f(end)$ non diventa negativo. Questo processo richiede un numero di iterazioni proporzionale a $\log n$, dato che ogni passo raddoppia l’indice end fino a superare il punto in cui f cambia segno.

Una volta identificati un indice end per cui $f(end) < 0$ e un indice $start$ per cui $f(start) \geq 0$ (andrebbe bene $start = 1$, ma sappiamo che $start = end/2$ è sicuramente positivo), procediamo con una ricerca dicotomica. L’obiettivo è ridurre l’intervallo tra $start$ e end finché end non è l’elemento immediatamente successivo a $start$, e dunque end sarà l’indice cercato, il più piccolo per cui $f(n) < 0$. Anche la ricerca dicotomica avrà costo computazionale $O(\log n)$.

```
int findNegative()
    int end = 1
    while f(end) ≥ 0 do
        end = end · 2
    return fnRec(end/2, end)
```

```
int fnRec(int start, int end)
    if end = start + 1 then
        return end
    int mid = (start + end)/2
    if f(mid) < 0 then
        return fnRec(start, mid)
    else
        return fnRec(mid, end)
```

Esercizio B1 – Connect-x – Punti ≥ 8

L’algoritmo può essere facilmente risolto in tempo $O(n^3)$, cercando la sequenza più lunga che parte da ogni casella nelle due direzioni.

Per risolvere il problema in tempo $\Theta(n^2)$, è possibile semplicemente scorrere ogni riga e ogni colonna, considerando la più lunga sequenza consecutiva che si incontra e aggiornando un variabile *maxSoFar* che verrà poi infine restituita. Potremmo chiamarla programmazione dinamica, ma direi che più semplicemente è buon senso.

```
longestLine(boolean[][] A, int n
```

```
    int maxSoFar = 0
    for i = 1 to n do
        int len = 0
        for j = 1 to n do
            if A[i][j] then
                len = len + 1
                maxSoFar = max(maxSoFar, len)
            else
                len = 0
        for j = 1 to n do
            int len = 0
            for i = 1 to n do
                if A[i][j] then
                    len = len + 1
                    maxSoFar = max(maxSoFar, len)
                else
                    len = 0
            return maxSoFar
```

Esercizio B2 – Formazione stringhe – Punti ≥ 10

Inizialmente, avevo pensato di utilizzare la programmazione dinamica. Sia $DP[i]$ il numero minimo di mosse per formare la stringa $S(i)$, ovvero il prefisso costituito dai primi i caratteri della stringa S . Ovviamente, $DP[1] = 1$, poiché la prima operazione possibile è aggiungere il primo carattere. Se i è pari, è possibile che $S(i)$ sia stata ottenuta duplicando $S(i/2)$ (ma ciò va verificato) oppure aggiungendo un carattere; in tal caso, si sceglierà il minimo fra i due casi. Altrimenti, se i è dispari, $S(i)$ è stato necessariamente ottenuto aggiungendo un carattere. La formula ricorsiva è quindi la seguente:

$$DP[i] = \begin{cases} 1 & \text{se } i = 1 \\ \min(DP[i-1] + 1, DP[i/2] + 1) & \text{se } i \text{ è pari} \wedge S[1 \dots i/2] = S[i/2 + 1 \dots i] \\ DP[i-1] + 1 & \text{altrimenti} \end{cases}$$

Il codice che implementa questa formula è il seguente:

```
int minMoves(item[] S, int n)
{
    int[] DP = new int[1...n]
    DP[1] = 1
    for i = 2 to n do
        if i mod 2 == 0 and check(S, i) then
            DP[i] = min(DP[i-1] + 1, DP[i/2] + 1)
        else
            DP[i] = DP[i-1] + 1
    return DP[n]
}
```

Per semplificare il codice, ho scritto una funzione $\text{check}(S, i)$ che controlla se il prefisso $S(i)$ è formato da due stringhe duplicate:

```
boolean check(item[] S, int i)
{
    for k = 1 to i/2 do
        if S[k] != S[k + i/2] then
            return false
    return true
}
```

Il costo dell'algoritmo è $O(n^2)$, in quanto dobbiamo riempire un vettore di n elementi, con costo $O(n)$ per metà di essi (quelli pari).

Poi mi sono reso conto che dovrebbe essere sufficiente utilizzare un semplice approccio greedy:

- Se la lunghezza n della stringa è dispari, non può derivare da un raddoppio: quindi l'ultimo carattere è stato aggiunto, contiamo una mossa e consideriamo la stringa di lunghezza $n - 1$.
- Se la lunghezza n della stringa è pari, verifichiamo se la sua prima metà è uguale alla seconda metà; se così non fosse, necessariamente l'ultimo carattere è stato aggiunto e contiamo una mossa, considerando il prefisso di lunghezza $n - 1$.
- Se la lunghezza n della stringa è pari e la prima metà è uguale alla seconda metà, la stringa può essere ottenuta con una mossa di raddoppio o con $n/2$ mosse di inserimento. La prima opzione sembra preferibile, ma bisognerebbe dimostrare che tale scelta greedy sia corretta.

In questo momento, non riesco a sviluppare una dimostrazione fatta bene. In generale, l'intuizione è la seguente: posticipando il raddoppio il più tardi possibile (e quindi utilizzando la divisione per due quando si parte dal fondo), si riduce il più possibile il numero di inserimenti singoli che devono essere effettuati.

Supponiamo per assurdo che esista una strategia più efficiente che preveda di raddoppiare prima rispetto all'approccio greedy. Questo significherebbe che c'è una configurazione in cui raddoppiare parzialmente (prima di raggiungere la fine) riduce il numero totale di mosse. Tuttavia, ogni raddoppio incompleto richiede successivamente inserimenti aggiuntivi per completare la stringa, il che contraddice l'ipotesi che tale strategia possa essere più efficiente.

Detto questo, non sono ancora totalmente soddisfatto della dimostrazione.

Il codice è il seguente:

```

int minMoves(item[], S, int n)
{
    int i = n
    int moves = 0
    while i > 0 do
    {
        if i mod 2 == 0 and check(S, i) then
        {
            i = i / 2
        }
        else
        {
            i = i - 1
        }
        moves = moves + 1
    }
    return moves
}

```

Il costo dell'algoritmo è comunque $O(n^2)$, in quanto nel caso pessimo non ci sono duplicazioni, ma dobbiamo comunque verificare se le varie stringhe sono duplicate, quindi ho scritto un codice più pulito ma che non è migliore dal punto di vista computazionale.

Appunti sulla correzione A questo punto mi trovo di fronte a un problema di valutazione: avevo una soluzione $O(n^2)$ in mente, molti studenti hanno trovato una soluzione $O(n^2)$ diversa dalla mia, ma non hanno dimostrato che è corretta. Vista la difficoltà dell'esercizio e i tanti modi con cui è stato affrontato l'esercizio, ho deciso di dare il 100% alla soluzione greedy, 90% in presenza di piccoli problemi. Ho comunque alzato il punteggio.

Esiste però un'altra soluzione greedy, che parte dall'inizio e raddoppia subito tutte le volte che è possibile. Questa soluzione non funziona, per esempio per "aaaaa"; si può ottenere come

- $\emptyset \rightarrow "a" \rightarrow "aa" \rightarrow "aaa" \rightarrow "aaaaa"$ (4 mosse)
- $\emptyset \rightarrow "a" \rightarrow "aa" \rightarrow "aaaa" \rightarrow "aaaaa" \rightarrow "aaaaa"$ (5 mosse)

Agli studenti che hanno utilizzato questo approccio, in una forma o nell'altra, è stato dato un voto sufficiente, perché comunque la struttura del ragionamento era buona; ma non posso prendere punteggio pieno.

A tanti altri esercizi molto complessi e difficili da interpretare è stato dato un voto insufficiente.

Esercizio B3 – Zigzagghiamo – Punti ≥ 12

L'algoritmo proposto utilizza un approccio di backtracking per esplorare tutte le possibili combinazioni di sottosequenze che rispettano la condizione di essere a zig-zag.

Nella funzione principale, viene inizializzato un vettore S che viene utilizzato per memorizzare le sottosequenze candidate; dopo di che, viene chiamata la funzione ricorsiva `zzRec()` per iniziare la generazione di sottosequenze. La funzione `zzRec()` prende in input i seguenti argomenti:

- A : l'array di input contenente gli interi;
- n : la lunghezza dell'array A ;

- S : il vettore che memorizza la sottosequenza corrente;
- i_A : l'indice corrente nell'array A da cui scegliere il prossimo elemento;
- i_S : L'indice corrente nell'array S in cui memorizzare l'elemento scelto;
- dir : la direzione dell'alternanza (1 per cercare un aumento, -1 per cercare una diminuzione).

Se $i_A > n$, cioè se tutti gli elementi dell'array A sono stati considerati, l'algoritmo stampa la sottosequenza $S[1 \dots i_S]$.

L'algoritmo può aggiungere l'elemento $A[i_A]$ alla sottosequenza S se:

- $i_S == 1$: questo è il primo elemento che viene aggiunto a S , quindi non ci sono vincoli;
- se $(A[i_A] - S[i_S - 1]) \cdot dir < 0$, allora la differenza tra l'elemento corrente $A[i_A]$ e l'ultimo elemento di S ha il segno opposto rispetto alla direzione (dir), rispettando così la condizione di zig-zag.

Se la condizione è soddisfatta, l'elemento $A[i_A]$ viene aggiunto a S , e la funzione `zzRec()` viene richiamata ricorsivamente con:

- i_A incrementato di 1 (per passare al prossimo elemento in A);
- i_S incrementato di 1 (per passare alla prossima posizione in S);
- dir invertito (per cambiare la direzione dell'alternanza).

L'algoritmo effettua anche una chiamata ricorsiva senza aggiungere l'elemento corrente, in modo da esplorare altre possibili sottosequenze senza di esso (e senza invertire la direzione).

L'algoritmo esplora tutte le possibili sottosequenze dell'array A . Nel caso peggiore, questo comporta una complessità esponenziale $O(2^n)$.

```
printZigZag(int[] A, int n)
    int[] S = new int[1...n]
    zzRec(A, n, S, 1, 1, 1)
```

```
zzRec(int[] A, int n, int[] S, int i_A, int i_S, int dir)
    if i_A > n then
        | print S[1...i_S - 1]
    else
        | zzRec(A, n, S, i_A + 1, i_S, dir)
        | if i_S == 1 or (A[i_A] - S[i_S - 1]) * dir < 0 then
            | | S[i_S] = A[i_A]
            | | zzRec(A, n, S, i_A + 1, i_S + 1, -dir);
```
